

Part I

ECE Themed

1 Close-loop Controller for DC Motor

Description

Simulate motor speed. Code a controller (by flexibly ingrating proportional(P), derivative (D) and integration (I) components) to reach target speed.

Classes

- Motor (private speed, inertia)
- Controller (private Kp,Ki,Kd)
- SimulationResult

Design Tips

- Design a class for each component in the closed loop control system, e.g. comparator, various gain elements (K_p , K_D , K_I , H).
- A separate class that represents the overall controller.
- A separate class for motor that models motor characteristics and environmental factors (e.g. load variations) and its effect on the actual speed.
- A separate output class that reads the controller output and the motor speed at regular intervals. Subclass it to make a CSVOutput class that dumps the readings into a CSV file for future plotting.
- Do the wiring as a part of the construction of controller object.

Stretch Goal

Can you make the entire system multi-threaded to show how the various elements (e.g. controller, motor and output) function concurrently?

Output

Export CSV for plotting in Excel

2 Digital Electronics Circuit Designer

Description

Build other gates, e.g. AND, OR, XOR from NAND/NOR gates. Build complex digital circuits, e.g. 4-bit counter, adder etc. by joining the gates. Demonstrate the working of the system with appropriate inputs and generating appropriately formatted outputs.

Classes

- Gates (NAND, NOR)
- Circuit
- SimulationResult
- Output/Display

Design Tips

- Design a class for each component in the digital circuit. Circuit elements should be created by connecting objects of more basic elements. Circuit should be created by connecting circuit elements.
- A separate output class that reads the circuit output. Subclass it to make a CSVOutput class that dumps the readings into a CSV file for future plotting.
- Do the wiring as a part of the construction of more complex circuit element objects.

Stretch Goal

Can you make the entire system multi-threaded to show how the various elements (e.g. gates and circuits) function concurrently?

Output

Export CSV for plotting in Excel

3 Design of a Circuit Construction and Simulation Library in C++ - MiniSPICE

Objective

To apply Object-Oriented Design to model physical electrical components and their interconnections. The library should allow a user to construct a circuit like real hardware and then simulate its DC and AC steady-state behaviour.

Core Idea to convey

A Resistor, Capacitor, Inductor are all "Components" but they behave differently in AC. A Circuit is a collection of Components connected via Nodes. Simulation is a separate responsibility from Construction.

Suggested Code Snippets

(Feel free to deviate if required, how do be ready to discuss your decisions.)

Component

```
class Component {
protected:
    std::string name;
    int nodeA, nodeB; // connected between which two nodes
public:
    Component(std::string n, int a, int b) : name(n), nodeA(a),
        nodeB(b) {}
    virtual ~Component() = default;

    virtual std::complex<double> getImpedance(double frequency)
        const = 0;
    virtual double getValue() const = 0; // R in Ohms,
        L in Henry, C in Farad
    virtual std::string getType() const = 0;

    int getNodeA() const { return nodeA; }
    int getNodeB() const { return nodeB; }
};

class Resistor : public Component {
private:
    double resistance;
public:
    Resistor(std::string n, int a, int b, double r) :
        Component(n,a,b), resistance(r) {}
    std::complex<double> getImpedance(double freq)
        const override {
        return {resistance, 0}; // Z_R = R
    }
    std::string getType() const override { return "R"; }
    double getValue() const override { return resistance; }
};

class Inductor : public Component {
private:
    double inductance;
public:
```

```

        std::complex<double> getImpedance(double freq)
        const override {
            return {0, 2 * M_PI * freq * inductance}; //  $Z_L = j\omega L$ 
        }
        //...
};

class Capacitor : public Component {
private:
    double capacitance;
public:
    std::complex<double> getImpedance(double freq)
    const override {
        return {0, -1.0 / (2 * M_PI * freq * capacitance)};
        //  $Z_C = 1/j\omega C$ 
    }
    //...
};

class VoltageSource : public Component {
private:
    double voltage; // For simplicity, ideal independent source
    double frequency; // 0 for DC
public:
    std::complex<double> getImpedance(double freq)
    const override {
        return {0, 0}; // Ideal voltage source has 0 internal
            impedance
    }
    //...
};

```

Circuit

```

class Circuit {
private:
    std::vector<std::unique_ptr<Component>> components;
    int numNodes;
    int groundNode; // reference node, typically 0
public:
    Circuit(int nodes) : numNodes(nodes), groundNode(0) {}

    void addComponent(std::unique_ptr<Component> comp) {
        components.push_back(std::move(comp));
    }

    // For inspection
    void listComponents() const;
}

```

```

    int getNumNodes() const { return numNodes; }
    const auto& getComponents() const { return components; }
};

```

Simulator

```

class ISimulator {
public:
    virtual ~ISimulator() = default;
    virtual std::vector<std::complex<double>>
        solve(const Circuit& circuit, double frequency) = 0;
};

// Simple Nodal Analysis implementation - can be naive
class NodalSimulator : public ISimulator {
public:
    std::vector<std::complex<double>>
        solve(const Circuit& circuit, double freq) override;
    // Returns node voltages
};

```

Analysis

```

class IAnalysis {
public:
    virtual ~IAnalysis() = default;
    virtual void run(Circuit& ckt) = 0;
};

class DCAnalysis : public IAnalysis {
    // Calls simulator with freq = 0
};

class ACAnalysis : public IAnalysis {
private:
    double startFreq, endFreq, step;
public:
    void run(Circuit& ckt) override;
    // sweeps frequency and stores results
};

```

Part II

DSAI Themed

4 Data Analytics Library

For a Data Analytics Library, don't just jump straight to writing functions - `mean()`, `filter()`, `plot()`. Shift them from procedural thinking to object-oriented thinking. Focus on the process of design, not just syntax.

Here is a framework you can use.

- **Dataset / DataFrame** - The central entity. It owns the data.
- **Column / Series** - A typed column inside a Dataset.
- **Analyzer / StatisticsEngine** - Does calculations, but does not own data.
- **Filter / Query** - Represents a condition.
- **Importer / Exporter** - Handles CSV, JSON I/O
Visualizer - Optional, for output

Suggested starter code:

```
// Abstract base - for Polymorphism
class ColumnBase {
public:
    virtual ~ColumnBase() = default;
    virtual std::string getName() const = 0;
    virtual size_t size() const = 0;
    virtual void print() const = 0;
};

// Templated concrete class - for Generic Programming + OOP
template<typename T>
class Column : public ColumnBase {
private:
    std::string name;
    std::vector<T> data; // Encapsulation: data is private
public:
    Column(std::string n) : name(n) {}
    void addValue(T val) { data.push_back(val); }
    // ... getters, etc.
};

class DataSet {
private:
    std::vector<std::unique_ptr<ColumnBase>> columns;
public:
```

```

        void addColumn(std::unique_ptr<ColumnBase> col);
        ColumnBase* getColumn(const std::string& name);
        void loadCSV(const std::string& filename);
        DataSet filterBy(const std::string& colName, auto condition);
    };

    // Strategy Pattern for Analytics
    class IAnalyzer {
    public:
        virtual ~IAnalyzer() = default;
        virtual double analyze(const ColumnBase& col) = 0;
    };

    class MeanAnalyzer : public IAnalyzer {
    public:
        double analyze(const ColumnBase& col) override;
    };
    class MedianAnalyzer : public IAnalyzer {
    public:
        double analyze(const ColumnBase& col) override;
    };

```

5 A Simple Artificial Neural Network Library in C++ - MiniANN

Objective

To understand and apply Object-Oriented Design to build a modular, extensible Artificial Neural Network from scratch. Efficiency is NOT the goal. Clarity of design is.

Learning Outcomes

- Abstraction of mathematical entities into classes
- Use of Inheritance, Polymorphism, Composition
- Separation of data, logic, and training strategy

Suggested Code Snippets

(Feel free to deviate if required, how do be ready to discuss your decisions.)

Activation Function

```

class IActivation {
public:
    virtual ~IActivation() = default;
    virtual double activate(double x) = 0;
    virtual double derivative(double x) = 0; // for backprop
};

class Sigmoid : public IActivation { ... };
class ReLU : public IActivation { ... };
class Tanh : public IActivation { ... };

```

Neuron

```

class Neuron {
private:
    std::vector<double> weights;
    double bias;
    std::unique_ptr<IActivation> activation;
    double last_output; // cached for learning
public:
    Neuron(int numInputs, std::unique_ptr<IActivation> act);
    double forward(const std::vector<double>& inputs);
    // Getters/Setters for weights - do not make data public
};

```

Layer

```

class Layer {
private:
    std::vector<Neuron> neurons;
public:
    Layer(int numNeurons, int numInputsPerNeuron,
          std::unique_ptr<IActivation> act);
    std::vector<double> forward(const std::vector<double>&
                                inputs);
};

```

Neural Network

```

class NeuralNetwork {
private:
    std::vector<Layer> layers;
public:
    void addLayer(const Layer& layer);
    std::vector<double> predict(const std::vector<double>&

```

```

        inputs);
    void train(const std::vector<std::vector<double>>& X,
               const std::vector<std::vector<double>>& y,
               int epochs, double learningRate);
};

```

6 KNN Classifier Library - MiniKNN

Concept: Classify a point based on its neighbours.

Suggested Code Snippets

(Feel free to deviate if required, how do be ready to discuss your decisions.)

```

class DataPoint {
    vector<double> features;
    string label;
    double distanceTo(DataPoint other);
}

class DataSet {
    vector<DataPoint> points;
    void loadCSV();
}

class CompositionIDistance {
    virtual double calculate() = 0;
}

class EuclideanDistance : public CompositionIDistance { ... }

class ManhattanDistance : public CompositionIDistance { ... }

class KNNClassifier {
    int k;
    unique_ptr<IDistance> metric;
    string predict(DataPoint p);
}

```

Dataset: Iris dataset - 150 rows only. Light domain knowledge.

7 K-Means Clustering - MiniCluster

Concept: Group unlabeled data.

Suggested Code Snippets

(Feel free to deviate if required, how do be ready to discuss your decisions.)

```
class Cluster {
    DataPoint centroid;
    vector<DataPoint> members;
}

class KMeans {
    int k;
    vector<Cluster> clusters;
    void fit(DataSet& ds);
}

class IInitialiser { ... }

class RandomInitialiser : public IInitialiser { ... }

class KMeansPlusPlusInitialiser : public IInitialiser { ... }
```

Dataset: Mall Customers or simple 2D points generated by student.
Deliverable: Print final centroids and cluster sizes.

8 Linear Regression from Scratch - MiniRegressor

Concept Predict a number, not a label.

Suggested Code Snippets

(Feel free to deviate if required, how do be ready to discuss your decisions.)

```
Column and DataSet - Reuse from your analytics library

class ILossFunction {
    virtual double compute() = 0;
}

class MSELoss : ILossFunction { ... }

class LinearModel {
    vector<double> weights;
    double bias;
    double predict();
    void train(GradientDescent trainer);
}

class Optimizer {
```

```
virtual void update() = 0;
} -> SGDOptimizerDataset: House prices with 2 features - area and price.
```

OOP Lesson: Model should not train itself, Optimizer does it.

9 Naive Bayes Spam Filter - MiniSpamFilter

Concept

Text classification

Suggested Code Snippets

(Feel free to deviate if required, how do be ready to discuss your decisions.)

```
class Document {
    string text;
    string label;
    map<string,int> wordCount;
}

class Vocabulary {
    set<string> words;
}

class NaiveBayesClassifier {
    void train(vector<Document>);
    string predict(Document);
}

class ITokeniser { ... }

class SimpleTokeniser : public ITokeniser { ... }

class StopWordTokeniser : public ITokeniser { ... }
```

Dataset

20 spam / 20 ham SMS in a .txt file.

OOP Lessons

Inheritance for tokenizers.

10 Perceptron / Simple ANN for Logic Gates - MiniPerceptron

Concept

The first neural network - learns AND/OR gate.

Suggested Code Snippets

(Feel free to deviate if required, how do be ready to discuss your decisions.)

```
IActivation -> StepActivation, Sigmoid

ActivationPerceptron {
    vector<double> weights;
    unique_ptr<IActivation> act;
}

PerceptronTrainer {
    void train(Perceptron&, DataSet, int epochs);
}
```

Dataset

AND gate truth table - 4 rows only.

Stretch Goal

If you finish early, make a Layer of 2 perceptrons for XOR.